AgentDescent: Gradient Descent Where the Parameters Are Agents

A Parallel, Asynchronous, Merge-Based Framework for Self-Evolving Agent Artifacts

Birfy Chen
birfychen@gmail.com
https://github.com/Birfy/agentdescent

August 2026

Abstract

Self-improving agent systems evolve their own skills, prompts, and harnesses, but largely through a *serial* loop — one candidate proposed, evaluated, and accepted per iteration — bounding improvement throughput at $O(1/T_{\text{iter}})$. We present **AgentDescent**, a framework that transplants the distributed-training stack onto agent self-evolution: artifacts as parameters, diffs with attached evidence as gradients, a version-vectored ledger as the parameter server, and merge decisions as optimizer steps. Because diffs, unlike gradients, do not add, aggregation is a discrete-space optimization: conflict resolution, fusion, and statistical acceptance under a per-artifact Beta posterior. Efficiency has three sources: overlap of I/O-bound rollouts; barrier-free execution under per-diff bounded staleness; and reduced validation cost, since merging k concurrent proposals requires a single held-out acceptance test. Eighteen published algorithms are ported as plug-ins and run unchanged under serial, synchronous, and asynchronous scheduling. On live models, synchronous parallelism yields a median 1.36× end-to-end speedup across eleven algorithms (11/11 improving, sign test p=0.001); on a gate-heavy workload it reaches a median 3.91× effective concurrency against a control measured at 1.00×, and barrier removal adds a further median 6.28× there while adding nothing in aggregate across the eleven (0.99×, p=1.00) — we characterize when it pays, and report that none of the speedups is accompanied by a measurable quality gain. An equal-budget comparison of merge-of-N against best-of-N selection does not separate the two, and instrumentation shows why: in both a single-key and a multi-key artifact the fusion step received at most one surviving diff, so merging barely ran — a condition our counters detect and future workloads must avoid. Code and reproduction commands for every number are open source.

1 Introduction

A growing body of work shows that large-language-model agents can improve their own artifacts: prompts [4, 7], in-context playbooks [40], skill libraries [31, 42], agentic workflows [11, 39], and their own code [25, 35, 38]. Most of these systems share an execution structure inherited from the Gödel machine [27]: a serial improvement loop in which one candidate modification is proposed, evaluated, and accepted or rejected before the next is considered. If one iteration takes T_{iter} wall-clock seconds, improvement throughput is bounded at one accepted change per T_{iter}, independent of available compute. Notable exceptions parallelize at the *population* level — FunSearch and AlphaEvolve run asynchronous samplers over island archives [24, 26] — but their candidates remain independent: concurrent work is selected among, not merged.

Distributed deep learning encountered the same bottleneck and resolved it with a now-standard stack: data-parallel workers computing gradients concurrently, parameter servers aggregating them [19], staleness-tolerant asynchronous updates [23], per-parameter adaptive steps [17], and gradient surgery for conflicting updates [36]. This paper develops the thesis that the same stack, suitably reinterpreted, applies to agent self-evolution — and identifies precisely where the reinterpretation must depart from the original.

AgentDescent (Fig. 1) instantiates the mapping. A library of evolvable artifacts plays the role of the parameter tensor; a diff annotated with an *evidence card* (the rollout that motivated it) plays the role of a gradient; a git-backed, version-vectored ledger plays the role of the parameter server; and the optimizer step is an *aggregator* that merges concurrent diffs. N workers roll out tasks against snapshots of the library and propose diffs in parallel; an asynchronous merger aggregates them, targeting $O(N/T_{\text{iter}})$ improvement throughput.

The mapping is productive precisely because of where it breaks. *Gradients add; diffs do not.* Two gradients can always be summed; two textual edits to the same artifact may be complementary, redundant, or contradictory. Aggregation is therefore not averaging but a decision procedure — staleness filtering, conflict resolution, fusion, and a statistical acceptance test (Section 4). A second disanalogy is equally consequential: training code is not modifiable by the gradients it computes, whereas a self-evolving system can in principle rewrite its own verifier. AgentDescent therefore enforces a layered governance model in which safety-critical components are frozen (Section 6).

Contributions.

1. **A system design and a discrete-space aggregator** mapping the distributed-training stack onto parallel self-evolution — per-diff bounded staleness, conflict projection, fusion, Beta-posterior acceptance, dual-branch promotion — with a formal account of where the analogy holds and where it must break (Sections 3 and 4).

2. **An efficiency analysis identifying three sources of speedup** (Section 5): rollout overlap, barrier removal, and — specific to merge-based evolution — reduced validation cost, in which merging collapses k acceptance tests into one and *reflective merge* extends the reduction to artifacts whose concurrent proposals conflict. The three do not compose unconditionally, and we characterize the regimes in which each pays.

3. **An extensibility mechanism** under which published algorithms are ported as plug-ins rather than forks (Sections 3.3 and 7): a diff-encoding `strategy` plus an optional `aggregator_factory`, or a declarative policy bundle. Eighteen ports run under all three schedulers without modification.

4. **A live-model evaluation** (Section 8) reported symmetrically: an eleven-algorithm scheduler study in which synchronous parallelism wins for all eleven methods (median 1.36× end-to-end) while barrier removal shows no aggregate further gain (0.99×), against a three-seed case study in which it reaches a median 6.28× on a gate-heavy workload — with the regime distinction analyzed; direct observation of the validation-cost reduction as falling model-seconds per rollout at a pinned budget; quality results under the most aggressive schedule; and an equal-budget merge-versus-selection comparison reported, with analysis, as not yet statistically separated.

2 Related Work

Self-evolving agents span prompt evolution [4, 7], context and playbook evolution [40], skill libraries [31, 42], workflow and harness search [11, 39], self-modifying code [25, 35, 38], evolutionary program search [18, 22, 24, 26, 28], and label-free self-play [2, 12, 41], with reflection [21, 29] as a shared inner proposal mechanism. Prompt optimization more broadly includes meta-prompting and search methods — APE [43], OPRO [33], DSPy [16], EvoPrompt [10] — and TextGrad [37], which also frames LLM feedback as a gradient analogue:

Table 1: The correspondence between distributed training and AgentDescent. The final row is a deliberate disanalogy, enforced rather than elided.

Model training	**AgentDescent**
parameter tensor θ	library of `Evolvable` artifacts
gradient g	`Diff` + `EvidenceCard`
parameter server	git-backed, version-vectored `Ledger`
optimizer step	`Aggregator` merge decision
per-parameter adaptive step (Adam)	per-artifact Beta-posterior acceptance
bounded staleness (async SGD/RL)	per-diff lag η + rebase re-verification
EMA / weight averaging	`dev`/`stable` dual branch
training code (not self-modifiable)	L0 frozen governance layer

TextGrad backpropagates textual critiques through a single computation graph, whereas AgentDescent's "gradients" are concurrent keyed diffs merged into one lineage under statistical acceptance. Population-based training [14] and the island architectures of FunSearch and AlphaEvolve parallelize by maintaining independent candidates and selecting among them; AgentDescent instead merges concurrent edits at the diff level. Most of the systems above publish a serial outer loop; AgentDescent treats that loop as one scheduling regime among three. On the systems side, AgentDescent borrows its vocabulary from distributed training: parameter servers [19], lock-free and bounded-staleness asynchrony [1, 8, 23], per-parameter adaptivity [17], gradient surgery [36], weight averaging [32], and tensor/pipeline parallelism [13, 30]. Concurrent work on parallel self-evolution (barrier-free evolution loops; co-evolutionary verification [3]) indicates that the throughput premise is an open hypothesis; our contribution is the specific synthesis — concurrent, staleness-bounded, conflict-resolved diff-level merge over a versioned ledger — together with the measurement harness required to evaluate it.

3 Problem Setup and System Design

3.1 Problem formulation

Let $\mathcal{A}$ denote the space of *artifact libraries*: finite keyed collections $A = \{a_k\}$ of versioned textual or executable artifacts (skills, prompts, playbooks, harness modules, file trees). An agent parameterized by A induces a policy π_A; given a task distribution $\mathcal{D}$ and a bounded reward $r \in [0, 1]$, the objective is

$$A^\star \;=\; \underset{A\in\mathcal{A}}{\arg\max}\; \mathbb{E}_{x\sim\mathcal{D}}\big[\, r(\pi_A(x),\, x)\big]\,, \tag{1}$$

optimized from rollout evidence only: $\mathcal{A}$ is discrete, and no gradient of r with respect to A exists. The serial baseline — propose one candidate A', evaluate on a held-out split D_{val}, accept if better — performs one accepted modification per iteration time T_{iter}. AgentDescent's premise is that with N concurrent proposers and merge-based aggregation, accepted-modification throughput can approach $O(N/T_{\mathrm{iter}})$ without sacrificing the acceptance discipline.

A *diff* d is a keyed partial edit over A, produced from a rollout by a *strategy* S that maps a natural-language proposal to operations on keys (rule slots, playbook bullets, file paths). The key structure is what makes two diffs comparable: edits on disjoint keys *fuse*; edits on the same key *conflict*. Every diff carries an *evidence card* $e = (x, \tau, r, v_{\mathrm{base}})$ recording the task, rollout, reward, and the library version the proposer observed.

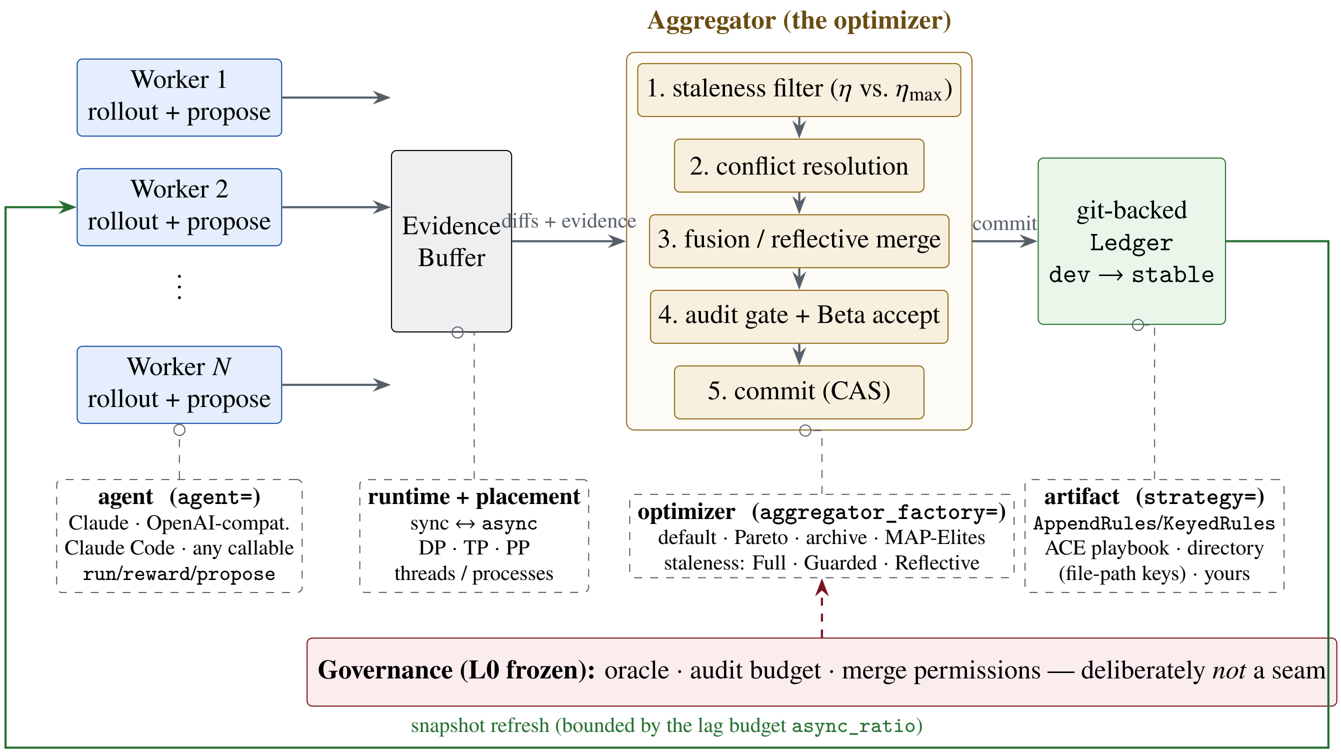

Figure 1: AgentDescent architecture. **Solid, top:** the fixed data path — N workers roll out tasks against ledger snapshots and emit diffs with evidence cards into a thread-safe buffer; a dedicated aggregator thread runs the five-stage merge pipeline and commits winners to the git-backed ledger; the green return edge is the only feedback path, bounded by the lag budget. **Dashed, bottom:** the pluggable seams of Section 3.3, each selected by one keyword argument of `evolve()`. **Red:** the frozen L0 governance layer, deliberately not a seam.

3.2 System components

Table 1 summarizes the correspondence and Fig. 1 the resulting architecture. The **ledger** is the parameter server: a git-backed store with per-artifact version vectors, compare-and-swap (CAS) commits, and a dual `dev`/`stable` branch pair; `stable` is promoted after K regression-free rounds on `dev`, an EMA-style confirmation in which a new commit restarts the counter. **Workers** hold snapshots of the library, roll out tasks, and emit (diff, evidence) pairs into a thread-safe buffer. The **aggregator** consumes the buffer and performs merge decisions (Section 4). Version vectors make the staleness of any diff computable at merge time:

$$\eta(d) \;=\; \max_{a \,\in\, \mathrm{touched}(d)} \big(v_{\mathrm{head}}(a) - v_{\mathrm{base}}(a)\big). \tag{2}$$

3.3 Pluggable seams

Each component in Fig. 1 sits behind an explicit interface, selected by one keyword argument of the single entry point `evolve()`: the *agent layer* (any `prompt` → `text` callable is a completion — Claude, OpenAI-compatible endpoints, a tool-using CLI agent, or plain `run/reward/propose` functions), the *strategy* (diff encoding), the *aggregator* (`aggregator_factory`), the *staleness policy*, the *runtime* (synchronous or barrier-free), the *parallelism scheme* (data/tensor/pipeline [13, 30]), and the *executor and sandbox*. Selection and acceptance rules are likewise named policy classes. Section 7 uses these seams to port eighteen published algorithms without modifying the engine. The single deliberate exception is governance (Section 6): the L0 layer exposes no seam, since a self-modifying system must not be able to replace its own evaluator.

4 The Aggregator as a Discrete-Space Optimizer

Evidence cards are bucketed per artifact; when a bucket reaches batch size B (or a timeout, so cold artifacts are not starved), the aggregator executes one optimizer step, summarized in Algorithm 1: **(1) staleness filtering** by Eq. (2) under the active policy (Section 5.2); **(2) conflict resolution** — syntactic (overlapping hunks) and semantic (contradictory operations) conflicts are detected, and contradictions are projected out in the manner of PCGrad [36], retaining the better member of each pair on a shared held-out subset; **(3) fusion** — complementary survivors are merged into one union candidate (cf. model soups [32]), with *reflective merge* handling contradictions that a keyed union cannot (Section 5.3); **(4) auditing** — merges with high blast radius or low trust are routed through an oracle with veto power; **(5) statistical acceptance and commit**. Let (s_c, f_c) and (s_b, f_b) be the candidate's and the incumbent's successes and failures on the held-out split D_{val}. The candidate commits (by CAS on `dev`) iff

$$\Pr[\theta_c > \theta_b] \;>\; 1 - \delta_v, \qquad \theta_c \sim \text{Beta}(1 + s_c,\, 1 + f_c), \quad \theta_b \sim \text{Beta}(1 + s_b,\, 1 + f_b), \tag{3}$$

with independent uniform priors and the probability estimated by Monte Carlo, matching the released implementation — a per-artifact posterior test rather than a point threshold, the counterpart of per-parameter adaptivity in Adam [17]. The confidence requirement δ_v is annealed over the artifact's version v (the learning-rate-decay analogue) and a trust region caps diff size.

One risk of this discipline deserves explicit statement: the gate reuses a single fixed D_{val} across many acceptance decisions (hundreds of proposals per run in Section 8.3), so accepted diffs are selected extreme statistics on D_{val} and their measured gains are optimistically biased — the standard adaptive-data-analysis concern. Two mitigations are in place and one is not: all quality results in Section 8 are reported on a disjoint *test* split the gate never sees; the trust region and annealed δ_v limit how much any single decision can exploit D_{val}; but no reusable-holdout or D_{val}-rotation mechanism is implemented. We therefore measure the resulting bias directly (Section 8.4): across 33 runs the gain the gate observed on D_{val} exceeds the gain realized on test by 0.038 on average, a small but statistically detectable optimism that a reader should subtract from any D_{val}-reported figure.

A natural objection to merge-based evolution is that two individually beneficial edits may be jointly harmful. Two properties bound the risk: the acceptance test Eq. (3) scores the *fused* candidate on the full held-out split, so a harmful fusion cannot commit; and the union is a superset of every constituent diff, so committing it discards no proposal. An optional *fusion tournament* additionally ranks the union against its constituents; it is disabled by default (it costs one additional held-out sweep per candidate against a recoverable gain) but serves as the measurement instrument for the objection, reporting win rate, the losing tail, and a `contested` counter that records whether a fused candidate was in fact constructed and ranked — so a run in which fusion never occurred cannot be reported as a merge win. We deliberately publish no headline win rate: the quantity is a property of the artifact's key granularity and of proposal overlap, not of the mechanism (Section 8.5 reports it in situ for one workload).

5 Efficiency Analysis: Three Sources of Speedup

Wall-clock time in a self-evolution loop decomposes into rollout time, synchronization time, and held-out validation time. AgentDescent addresses each with a distinct mechanism; the third is specific to merge-based evolution.

5.1 Overlapping I/O-bound rollouts

Agent rollouts are dominated by time spent waiting on a model endpoint, so thread-level concurrency suffices to overlap them (Section 8.1 measures 5.8× at eight threads on real API calls). The synchronous data-parallel runtime shards tasks across N workers over a common snapshot and merges at a round barrier. The barrier, however, imposes a ceiling independent of N: with i.i.d. rollout latencies t_i,

$$T_{\text{round}}^{\text{sync}} \;=\; \mathbb{E}\big[\max_{i \le N} t_i\big] \;+\; T_{\text{gate}}, \tag{4}$$

and reasoning-model latencies are heavy-tailed — a short answer and a long deliberation are the same call — which maximizes the gap between $\mathbb{E}[\max_i t_i]$ and $\mathbb{E}[t]$. The second term is a serial region in which all workers idle while the gate evaluates.

5.2 Barrier-free execution under bounded staleness

The asynchronous runtime (Algorithm 1) removes the round structure: workers produce continuously, a dedicated merger consumes continuously, and per-rollout cost approaches $\mathbb{E}[t]$ — the no-barrier discipline of asynchronous RL systems [1, 8] transferred to self-evolution. Its cost is staleness, which Eq. (2) makes first-class and per-diff, bounded by two mechanisms. A *lag budget* ρ (`async_ratio`) limits drift at the source: a worker refreshes its snapshot once v_{head} exceeds its snapshot version by more than ρ, and backpressure forces a global resynchronization if evidence accumulates while nothing commits. A *staleness policy* disposes of stale diffs at the merger: Full admits them unconditionally (sound only where diffs compose); Guarded rebases within $\eta \le \eta_{\max}$ and discards beyond, in the spirit of bounded-staleness RL [8], paying for safety in discarded rollouts; Reflective always rebases and *re-verifies* — a stale diff is treated as unproven rather than invalid, and is discarded only if its claimed improvement fails to hold on the new head. Because ρ is denominated in artifact versions, its appropriate value scales with rollout cost; the runtime emits a warning when a run discards the majority of its evidence.

5.3 Reducing validation cost

The third source of speedup is unavailable to the first two mechanisms: the gate itself. Every acceptance decision costs one sweep over D_{val}, and on live models this dominates the merger's critical path: profiled on a live GEPA/HotpotQA run (N=4, `GLM-5.2`), the merge thread's gate time *equals* its total busy time (560.1 s of each within a 740 s process, i.e. merger occupancy ≥ 76%, all of it evaluation). A serial or selection-based loop with m proposals pays $m \cdot |D_{\text{val}}|$ gate evaluations. Merging changes the count: a round's k surviving diffs fuse into one candidate, so per-round gate cost falls from $k \cdot |D_{\text{val}}|$ to $|D_{\text{val}}|$; a diff discarded by the staleness filter never reaches the gate and costs zero evaluations; and in the asynchronous runtime, cards arriving during a merge enlarge the next batch rather than triggering additional merges. The effect is visible directly in the RQ1 experiment (Fig. 3): model-seconds per rollout fall 37.9 → 27.7 → 23.8 across the serial, synchronous, and asynchronous arms at a pinned rollout budget. Two costs partially offset the saving and are stated for balance: the Reflective staleness policy's re-verification spends additional gate evaluations, and a model-written reflective merge spends one further model call per contested fusion. Residual gate cost parallelizes independently of the worker pool (`eval_concurrency`), saturating at $|D_{\text{val}}|$.

The reduction presupposes that proposals *can* fuse, which fails exactly when the artifact's key space is too coarse. An artifact held in a single key — e.g. a prompt as one instruction slot, GEPA's setting [4] — makes every pair of concurrent proposals a write–write conflict; a keyed union then degenerates to retaining the better single proposal, i.e. best-of-n selection at best-of-n validation cost. *Reflective merge* restores the reduction: a model is prompted to write the *union* of the contradicting proposals — one candidate preserving each proposal's intent — and the union passes through the same acceptance test as any candidate, so the model proposes and the gate decides. Key granularity is necessary but not sufficient: Section 8.5 finds that on a multi-key artifact the fusion step is often handed a *single* surviving diff, in which case there is nothing to fuse regardless of how the keys are laid out. It is the spatial counterpart of the Reflective staleness policy (carry

Algorithm 1 Barrier-free evolution (`async_evolve`); the synchronous runtime is the special case with a round barrier between the two loops. S is the strategy of Section 3.1. Backpressure has two parts: `buffer.put` blocks while the un-merged intake exceeds the lag budget, and a stall detector (evidence arriving while nothing commits) forces a global resynchronization. In the current implementation a single merger thread performs all commits, so the CAS never contends; multi-merger operation, where it would, is future work.

Worker i **(in parallel,** $i = 1..N$**):**
loop
  if $v_{\text{head}} - v_i > \rho$ **then** refresh snapshot $A_i \leftarrow$ ledger head; $v_i \leftarrow v_{\text{head}}$
  end if
  sample task x; rollout $\tau \leftarrow \pi_{A_i}(x)$; reward $r \leftarrow r(\tau, x)$
  $(d, e) \leftarrow S.\text{propose}(A_i, x, \tau, r)$; buffer.put$(d, e)$ ▹ blocks on backpressure
end loop
Merger (single thread):
loop
  $\mathcal{B} \leftarrow$ buffer.drain(); bucket $\mathcal{B}$ by artifact
  for each bucket with $|\mathcal{B}_a| \geq B$ or timed out **do**
    $\mathcal{B}_a \leftarrow \{\, d : \text{policy}(\eta(d), \eta_{\max}) \neq \textsc{discard} \,\}$, rebasing where directed ▹ Eq. (2)
    resolve conflicts (keep better of each contradicting pair); $u \leftarrow$ fuse(survivors)
    if survivors contradict on a key **and** reflective merge enabled **then** $u \leftarrow$ model-written union
    end if
    if audit passes **and** Eq. (3) holds on D_{val} **then** CAS-commit u to `dev`
    end if
  end for
  promote `dev` $\to$ `stable` after K regression-free rounds
end loop

intent onto a changed base; let measurement decide), and the `contested` counter reports whether fusion genuinely occurred, so the saving is never claimed where only selection took place.

6 Governance by Blast Radius

Artifacts are assigned to layers by a declared `blast_radius`: **L2** (skills, prompts) merges under the full asynchronous pipeline; **L1** (harnesses, verifiers) serializes in-flight changes and routes every merge through the oracle; **L0** (the oracle, audit budget, merge permissions, safety constraints) is frozen and read-only to the loop. The frozen layer enforces the second disanalogy of Table 1: without it, a self-referential loop can converge to a verifier that accepts its own output. The same layering protects executed agent code, which runs behind a frozen test suite the candidate cannot rewrite; pristine tests are overlaid after workspace materialization, so weakening them at run time is likewise ineffective.

7 Case Study: Porting Eighteen Self-Evolution Algorithms

Because every component sits behind a seam (Section 3.3), porting a published algorithm requires a small set of plug-ins rather than a fork. Two routes cover all eighteen ports.

Route 1: a strategy, plus an optimizer where needed. A *benchmark-faithful* port supplies a `strategy` encoding the paper's artifact as keyed diffs and, only when its search differs from the default pipeline, an `aggregator_factory`. The seven faithful ports illustrate the scale of the required code: ACE [40] is a playbook strategy whose curator *is* the default aggregator; GEPA [4] substitutes a per-instance Pareto optimizer; ADAS [11] and DGM [38] plug in keep-all archives with their own parent selection; EvoSkill a bounded top-K frontier; SkillOpt a strict edit gate;[1] OpenEvolve [28] MAP-Elites islands [22]. Parent-selection and acceptance rules are extracted as named policy classes, so subsequent ports reuse them.

Route 2: a declarative policy bundle. The eleven microports and analogues (PromptBreeder, AFlow, Self-Refine, Reflexion, SICA, Gödel Agent, Voyager, SkillWeaver, Absolute Zero, R-Zero, Agent0) define no new classes: each is a `MethodPolicy` — selection rule, memory, acceptance shape, genome — over one shared runner, inheriting a common budget contract, dataset layer, and command line. A shared `--serial` flag reproduces the upstream algorithm's own semantics (one worker, serial gate) as the control that any parallelization claim requires, and a shared rollout budget pins compared arms to equal work.

Fidelity is declared per port — *benchmark-faithful*, *mechanism microport*, *self-edit analogue*, *environment analogue*, or *inference analogue* — follows the released code where it diverges from the paper, and documents infrastructure boundaries explicitly; analogues are not to be cited as benchmark reproductions. All eighteen ports run under serial, synchronous, and barrier-free scheduling without modification, which is what permits the controlled scheduler comparison of Section 8.2.

8 Evaluation

Setup. All results are obtained from live models on real datasets: `deepseek-v4-flash` and `GLM-5.2` (reasoning models) via an OpenAI-compatible endpoint, on each algorithm's own benchmark. Every number is produced by a named script in the repository with its exact reproduction command; absolute times depend on host and endpoint, so ratios are the reproducible quantity. We address four questions: **RQ1** — how much effective concurrency do the three mechanisms of Section 5 deliver against a faithful serial control? **RQ2** — does the speedup generalize across algorithms? **RQ3** — is learning behavior preserved under the most aggressive schedule? **RQ4** — does merging outperform best-of-N selection at equal budget?

8.1 RQ1: Effective concurrency against a faithful serial control

We first verify the premise of Section 5.1: eight threads over one pool reduce a real `GLM-5.2` API workload from 48.6 s to 8.3 s (5.8× in this run; 5.8–6.5× across three repeats, sub-linear under heavy-tailed latency), while the same threads on pure-Python CPU work yield 1.1× — rollout overlap is real, and the GIL does not limit I/O-bound rollouts.

The main experiment ports GEPA to its own benchmark (HotpotQA [4, 34]) with 16 rollouts pinned on every arm, three seeds, and a control that reproduces the upstream loop exactly: one worker *and* a serial gate. That control lands at 0.99, 1.00, and 1.00× overlap on the three seeds (Fig. 2), which is worth stating on its own: the property that makes it a faithful control is stable, not a fortunate single run. Against it, synchronous parallelism spans 3.18–3.96× effective concurrency (median 3.91) and barrier removal spans 3.30–6.44× (median 6.28); median wall-clock falls 790 → 152 → 116 s. Figure 3 shows the validation-cost mechanism of Section 5.3 in the same runs, and it replicates across seeds: model-seconds per rollout fall from a median 49.3 (serial) to 37.6 (synchronous) to 36.7 (asynchronous) at the pinned budget — the parallel arms not only

[1] EvoSkill and SkillOpt are ports of released systems whose upstream papers and per-port departures are identified in the repository's port-fidelity documentation; we do not cite them here to avoid attributing a reconstruction to the wrong reference.

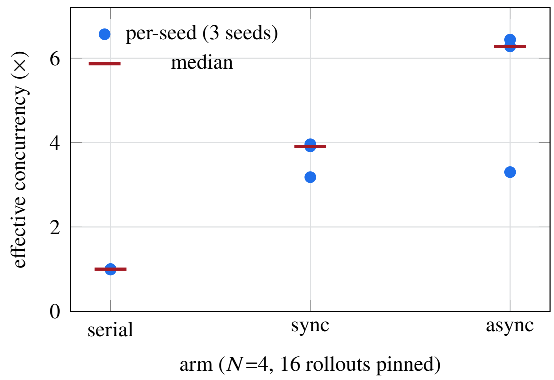

Figure 2: Effective concurrency (model-seconds ÷ wall-clock), GEPA on HotpotQA, `GLM-5.2`, 16 rollouts pinned per arm, three seeds. The serial control reproduces the published loop (one worker, serial gate) and lands at 0.99–1.00× on every seed — the control property is a stable one, not a single-run coincidence. Synchronous parallelism spans 3.18–3.96×; barrier removal spans 3.30–6.44× but is by far the widest arm, and it reaches those figures while overrunning the pinned budget (19 rollouts against 16).

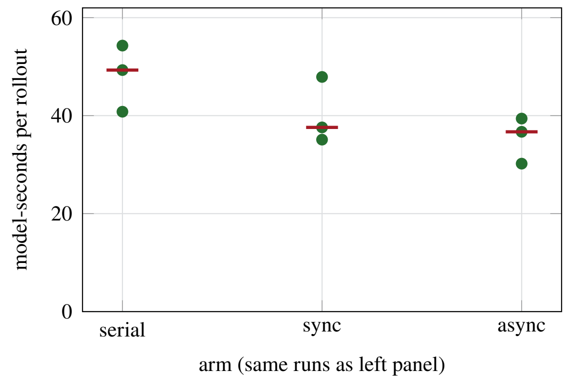

Figure 3: Validation-cost reduction in the same runs (Section 5.3): model-seconds spent per rollout at a pinned rollout budget, per seed, with medians marked. The parallel arms evaluate less per unit of work — median 49.3 → 37.6 → 36.7 — because reflective merge collapses each round's admissions into one gate sweep and stale proposals are discarded before they are scored.

finish sooner, they evaluate less per unit of work. Independent endpoint probes sustain 7× (matched width and request length) to 10× (wider, shorter requests) concurrency, so no arm is provider-limited.

Three qualifications travel with these numbers, and they matter more than the headline. First, *the asynchronous arm is not paying the same bill*: lacking a round boundary it ran 19 rollouts against the pinned 16 on every seed, so part of its concurrency is work the other arms did not perform. Second, *it is also the least stable arm* — a 3.30–6.44× spread against synchronous parallelism's 3.18–3.96×. Third, and most important, *none of this is a quality claim*: median held-out test score is 0.55 (serial), 0.60 (synchronous), 0.50 (asynchronous) across the three seeds, on a 20-item split whose resolution is 0.05, so the arms are indistinguishable in quality and the fastest arm is not the best one. One synchronous cell (seed 0) also ended early on an endpoint error at 12 of 16 rollouts and is included as measured.

These figures supersede a single-run measurement of 1.85 × /3.22× on a different model reported in an earlier draft. Two differences account for the gap and neither is a correction of the other: this run uses `GLM-5.2` rather than `deepseek-v4-flash`, and its parallel arms run the gate at `eval_concurrency=16`. The second is the more instructive: with the gate itself parallelized, the round barrier costs much less, which both raises the synchronous arm and shrinks what barrier removal has left to recover — exactly the regime dependence that Section 8.2 finds across eleven algorithms. An earlier draft additionally reported a width-8 variant with a 28% model-call saving; it is withdrawn because its serial arm ran with a concurrent gate (overlap 1.31×), violating the control definition above.

8.2 RQ2: Generality across algorithms

To test whether the RQ1 result generalizes beyond GEPA, the same scheduler comparison — serial, synchronous, barrier-free — is applied to all eleven declarative ports on `GLM-5.2` at an equal candidate budget and width 2, varying only the scheduler (three paired seeds per method; 99 observations). The results cut both ways, and we report both. *Synchronous parallelism generalizes*: paired over n=33 runs it outperforms the serial arm by a median 1.36× end-to-end and 1.89× within the engine window, with a 1.47× median

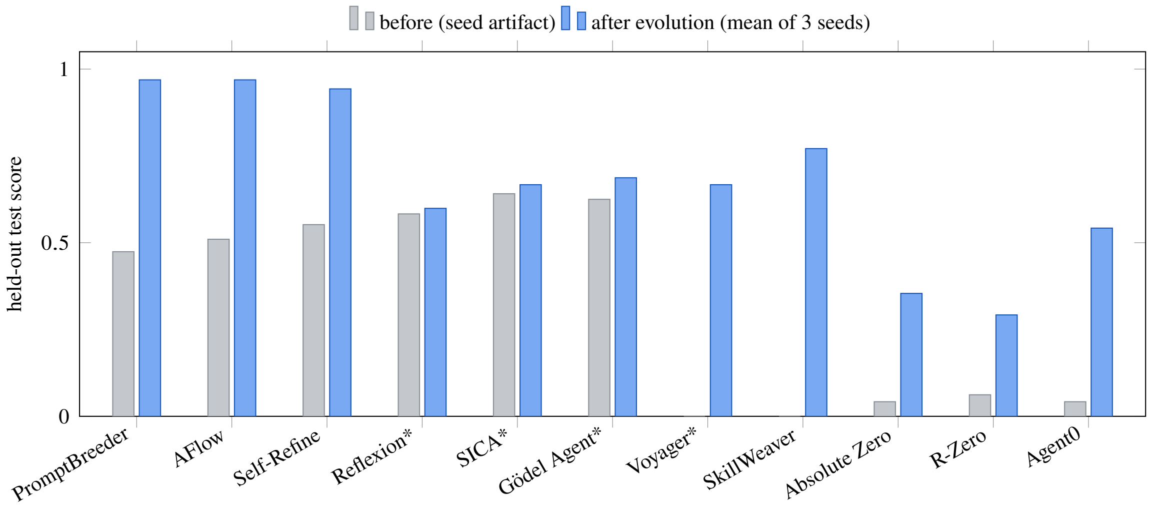

Figure 4: Held-out test score before and after evolution for the eleven microports and analogues, each run against `deepseek-v4-flash` under the *most aggressive* configuration — barrier-free async pipeline, 8 workers, Full staleness, 80 rollouts per seed, 3 seeds (465–2,069 model calls per seed). "After" bars are three-seed means; asterisked methods improved on two of three seeds (the rest on all three), and Reflexion's gain is within its domain's ±0.02 noise floor. Between 3 and 10 of the 240 proposals per method (80 × 3 seeds) commit — the gate admits only strict held-out improvements, so sparse acceptance is the shape of the mechanism, not a low yield. Bars are comparable only within a domain; per-seed spreads are tabulated in the repository documentation.

time-to-quality (n=18 seeds reaching the quality bar). The win is unanimous across methods — 11 of 11 on both the end-to-end and engine-window medians, a two-sided sign test giving $p = 0.001$ in each case. *Barrier removal does not*: the asynchronous arm aggregates to 0.99× end-to-end and 0.97× engine-window relative to synchronous, and the per-method record is indistinguishable from chance — 5 methods faster, 5 slower, 1 tied end-to-end ($p = 1.00$), and 2 faster against 8 slower in the engine window ($p = 0.11$, i.e. if anything a slight cost). Its one positive signal is time-to-quality, a modest 1.10× median (n=15) concentrated in a few methods (SkillWeaver 1.25×, Self-Refine 1.24×, Reflexion 1.12×). The reconciliation with RQ1's 3.22× is the regime: barrier removal pays in proportion to how much heavy-tailed rollout latency and gate cost there is to hide, which GEPA's long rollouts and expensive Pareto gate maximize and these compact ports mostly lack; asynchrony helps time-to-quality only when a useful commit can land before a synchronous barrier would. Two caveats: this matrix was measured on the implementation preceding the ports' declarative restructuring (mechanisms and budgets unchanged by design; a rerun on current code is pending), and its per-run JSON is retained as a summarized report rather than raw. We report it despite the caveats because it is the only multi-algorithm evidence in either direction, and it bounds the asynchronous claim honestly.

8.3 RQ3: Quality preservation under the aggressive schedule

Throughput is meaningful only if learning is preserved, so quality is measured under the most aggressive configuration (barrier-free, 8 workers, Full staleness). Figure 4 reports the eleven declarative ports at three seeds each. Read precisely: ten of eleven improve their three-seed mean test score beyond their domain's noise floor; Reflexion's +0.016 on GSM-Hard [5, 9] sits within that domain's ±0.02 floor and should be read as no change; and four methods (Reflexion, SICA, Gödel Agent, Voyager) improve on two of three seeds rather than all three. Gains are bounded by each benchmark's headroom — the three GSM8K ports converge to 0.94–0.97 and differ in cost rather than quality. What this establishes is that the aggressive schedule does not stop the ports from learning; a paired serial-schedule quality arm at equal rollout budget, which would establish that asynchrony costs no quality *relative to serial*, has not been run and we do not claim it. Among the benchmark-faithful ports, three representative single-seed results establish end-to-end operation (not effect sizes): ACE improves FiNER-139 [20] validation 0.719 → 0.766 over 120 rollouts while curating a ten-bullet playbook; DGM's best archived child repairs held-out vendored bugs at 0.906 versus its seed agent's 0.844 under a frozen `pytest` oracle (a compact SWE-bench-style setting [15]); SkillOpt improves a hard SearchQA [6] subset 0.053 → 0.211 with 3 of 43 proposed edits accepted. One port produced no quality number at all: ADAS's benchmarks were either saturated (MGSM) or cost-prohibitive per call (GPQA), and we state this rather than average over it. The highest-level interface (`evolve_skill`) on 40 HotpotQA rows raises held-out exact match from 0.167 to 0.583 over 338 model calls, committing one proposal.

8.4 RQ3b: How optimistic is the gate? The val–test gap

The acceptance test reuses one fixed D_{val} across every decision, so an accepted diff is a selected extreme statistic on that split and its measured gain should be optimistically biased (Section 4). The size of that bias is measurable from the runs already reported: each of the 33 runs behind Fig. 4 records both its D_{val} trajectory and its disjoint test score, so the per-run difference (test gain − val gain) estimates the optimism directly. Mean val gain is +0.395 and mean test gain +0.357, a mean gap of **−0.038** (median −0.031, s.d. 0.075); test gain falls short of val gain in 20 of 33 runs and exceeds it in 7, with 6 exact ties — a two-sided sign test on the 27 non-tied runs gives $p = 0.019$. The bias is thus real and directionally as predicted, but small relative to the gains themselves (roughly a tenth of the mean gain), and it does not threaten the sign of any result in Fig. 4, whose scores are reported on test throughout. The runs where it is largest are those with the coarsest reward signal (SkillWeaver, −0.125 to −0.250 on an eight-task test split), which is the expected place for a selected statistic to overfit.

8.5 RQ4: Merge-of-N versus best-of-N selection at equal budget

Throughput cannot distinguish merging from sampling-and-selecting, since N independent forks occupy N workers equally well; the discriminating quantity is held-out quality at an equal rollout budget. The largest such run to date — HotpotQA on `GLM-5.2`, three arms × three seeds at nine rollouts per seed, budget pinned in rollouts, reflective merge enabled so that contradicting proposals survive to fusion; 1,298 model calls (29 failing at the endpoint) and 11.3 hours of model time — yields a **null result**: merge-of-3 attains the highest median test score together with the widest spread, the intervals overlap, and the framework's own separation test declines to declare a winner. The diagnosis is more informative than the verdict: at this budget the fused union was constructed twice in the entire experiment — most merges saw a single surviving diff — so the experiment does not measure a merging deficit; it measures a regime in which merging rarely fired. It also confirms the key-granularity condition of Section 5.3 from the opposite direction: without reflective merge, the merge arm on this single-key artifact would have reduced to best-of-N selection. We therefore tested that diagnosis directly on a *multi-key* artifact, where the granularity argument predicts fusion should fire: ACE's playbook on FiNER-139, whose state is keyed per bullet, so two workers editing different bullets compose by construction. The prediction failed, and the way it failed is the more useful result. Over a 24-rollout, four-worker run (129 model calls), the aggregator held **2 fusion tournaments and contested none of them**: one had a *single candidate* and none had contradicting candidates. Fusion did not fire, and not for the reason the single-key analysis identified — there was nothing to contradict, because there was rarely more than one surviving diff to merge at all.

Read together, the two experiments locate the binding constraint one level below key granularity. A round's diffs reach the fusion step only after the staleness filter and conflict resolution have run, and at these budgets that pipeline typically leaves *one* survivor: on the single-key artifact because contradicting proposals are resolved down to the better one, and on the multi-key artifact because few proposals are produced per round in the first place. Merging cannot beat selection in a regime where it is handed one candidate, and no amount of key-space refinement changes that. What would is a higher proposal survival rate per merge — more workers per round, a weaker per-round filter, or budgets large enough that several diffs coexist — and that, rather than a larger version of the present experiment, is the measurement the question now needs.

Two caveats bound this run specifically. It is one seed, and it is run on a workload with little room to move: the seed playbook already scores 0.842 on test and 0.938 on the gate's split, and the run ended at test 0.842 — unchanged — having committed one proposal and rejected another. A saturated workload is a poor place to compare quality, and we make no quality claim from it; the fusion counters are structural observations about what the merge step was handed, and those do not depend on headroom. The repository's only multi-key benchmark is this one, so a workload that is both multi-key and unsaturated for a current model is a prerequisite for the definitive experiment, and building one is future work.

9 Limitations

AgentDescent is a research reference implementation, and its evaluation has stated boundaries. (i) *The central question is unresolved, and now for a sharper reason*: merging is not shown to beat best-of-N selection at equal budget, because in both settings measured the fusion step was handed at most one surviving diff (Section 8.5). Resolving it needs a workload that is multi-key *and* unsaturated for the model under test, plus a configuration that keeps several proposals alive per merge; the repository currently supplies neither. (ii) *Statistical strength*: the RQ1 ladder and the eleven-port quality results carry three seeds and are reported as spreads; the benchmark-faithful ports carry one seed each; the thread-overlap figure spans 5.8–6.5× over three repeats. Quality splits are small enough (20 items in RQ1) that no arm's quality is resolvable, which is why speed and quality are reported separately and no arm is claimed to be free. These establish that the mechanisms operate and in which direction, not effect sizes. (iii) *Scale*: the system is a single Python process — one merger thread (measured at $\geq$ 76% occupancy at N=4, all of it gate evaluation), thread workers, a local git ledger. Multi-machine operation does not exist, the CAS commit path never contends under a single merger, no worker sweep beyond N=8 has been run, and there is no failure-recovery story beyond endpoint retries; a merger-saturation model is future work. (iv) *Holdout reuse*: the gate reuses one fixed D_{val} across all acceptance decisions (Section 4); test splits are disjoint, but no reusable-holdout mechanism exists and the val–test gap of accepted diffs is not yet reported. (v) *Pending reruns*: the RQ2 matrix predates a port restructuring and awaits a rerun; turn-level checkpoint-resume of stragglers and a stratified tail-canary split are designed but unimplemented; difficulty parameters calibrated on one model did not transfer under a model swap, whereas benchmark-intrinsic difficulty did. (vi) *Correction trail*: several earlier measurements proved irreproducible or favorably defined and were re-measured and corrected in place — including, in an earlier draft of this paper, a width-8 experiment withdrawn for a flawed control and a misattributed per-rollout statistic (Section 8.1). Every number reported here is generated by a named script. We regard this trail as part of the contribution: the evaluation of a self-improving system should satisfy the standard its own acceptance gate imposes on diffs.

10 Conclusion

The distributed-training stack admits a coherent counterpart in agent self-evolution once its necessary disanalogy — diffs do not add — is taken seriously and merging is constructed as a discrete-space optimizer. Efficiency decomposes into three sources with different evidential status: rollout overlap and synchronous parallelism generalize across algorithms (median 1.36× end-to-end over eleven ports, all eleven improving); barrier removal is a regime-dependent win, reaching a median 6.28× effective concurrency on a gate-heavy workload while showing no aggregate gain across eleven compact ones; and reduced validation cost — merging collapses k acceptance tests into one — is visible directly as falling model-seconds per rollout at a pinned budget. Eighteen published algorithms adapt as plug-ins rather than forks. Whether merging also improves quality over best-of-N selection at equal budget remains open, and our measurements sharpen why: in both a single-key and a multi-key setting the fusion step was handed at most one surviving diff, so the mechanism under test barely ran. The instruments to detect that condition ship with the framework, and the workload that would avoid it — multi-key, unsaturated, and configured to keep several proposals alive per merge — is the concrete next step.

Acknowledgments

The algorithm ports were contributed by `chendanyang` (the seven benchmark-faithful ports) and `cyanneko` (OpenEvolve and the eleven microports and analogues).

Reproducibility

All code is available at https://github.com/Birfy/agentdescent (MIT license; `pip install agentdescent`; the core engine has no required dependencies). Generating scripts, per result: thread overlap — `examples/efficiency.py --only gil`; RQ1 and the live merger profile of Section 5.3 — `bench/matrix_run.py --rows gepa`; RQ2 — `bench/candidate_methods.py`, measured on the pre-restructuring implementation (summarized report retained, raw per-run JSON not; rerun pending); RQ3 — each port's own command line over `examples/_method_runner.py`, the entry point that accepts the `--staleness full` configuration used, with the resolved configuration printed as one JSON line per run; the `evolve_skill` case — `examples/skill_evolution.py`; RQ4 — `bench/baselines_run.py`. Every port supports `--dry-run` (no API key or model call) and carries an offline test suite; datasets are retrieved from canonical sources by a dependency-free data layer.

References

[1] ROLL Flash: Accelerating RLVR and agentic training with asynchrony. arXiv preprint, 2025. Preprint; identifier omitted pending verification.

[2] Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. arXiv preprint, 2025. Preprint; identifier omitted pending verification.

[3] Self-evolving agent skills via co-evolutionary verification (CoEvoSkills). arXiv:2604.01687, 2026. Concurrent work; official code unreleased at the time of writing.

[4] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

[5] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[6] Matthew Dunn, Levent Sagun, Mike Higgins, V Ugur Guney, Volkan Cirik, and Kyunghyun Cho. SearchQA: A new Q&A dataset augmented with context from a search engine. *arXiv preprint arXiv:1704.05179*, 2017.

[7] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.

[8] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.

[9] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning*, pages 10764–10799, 2023.

[10] Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations*, 2024.

[11] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

[12] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. *arXiv preprint arXiv:2508.05004*, 2025.

[13] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[14] Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, Chrisantha Fernando, and Koray Kavukcuoglu. Population based training of neural networks. *arXiv preprint arXiv:1711.09846*, 2017.

[15] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv preprint arXiv:2310.06770*, 2023.

[16] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.

[17] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[18] Joel Lehman, Jonathan Gordon, Shawn Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O Stanley. Evolution through large models. In *Handbook of Evolutionary Machine Learning*, pages 331–366. Springer, 2023.

[19] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 583–598, 2014.

[20] Lefteris Loukas, Manos Fergadiotis, Ilias Chalkidis, Eirini Spyropoulou, Prodromos Malakasiotis, Ion Androutsopoulos, and Georgios Paliouras. FiNER: Financial numeric entity recognition for XBRL tagging. In *Proceedings of ACL*, 2022.

[21] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[22] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.

[23] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent. In *Advances in Neural Information Processing Systems*, volume 24, 2011.

[24] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

[25] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.

[26] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M Pawan Kumar, Emilien Dupont, Francisco J R Ruiz, Jordan S Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[27] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.

[28] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. https://github.com/codelion/openevolve, 2025.

[29] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[30] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[31] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

[32] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998, 2022.

[33] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024.

[34] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.

[35] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.

[36] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.

[37] Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic "differentiation" via text. *arXiv preprint arXiv:2406.07496*, 2024.

[38] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[39] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.

[40] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.

[41] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.

[42] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.

[43] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*, 2023.